

FUNCTION(INTERVIEW QUESTION)

Python Functions: Conceptual and Coding Questions

Part 1: Conceptual Questions (1-30)

1. What is a function in Python, and how is it different from a method conceptually and internally?
2. Why does Python treat functions as first-class objects? What advantages does this provide?
3. What happens in memory when you define a function using `def`? Is the body executed at that time?
4. What is the difference between parameters and arguments? Explain with examples.
5. What are positional arguments and keyword arguments? What rules does Python enforce on their order?
6. What are default arguments? When exactly are default values evaluated?
7. Why is using a mutable object (like a list) as a default argument dangerous? How do you fix it?
8. What is `*args`? How does Python collect extra positional arguments internally (as which data type)?
9. What is `**kwargs`? How does it differ from `*args`?
10. What is the difference between `*` and `**` in a function definition versus in a function call?
11. What are keyword-only arguments? Explain the role of `*` in the parameter list.
12. What are positional-only arguments? Explain the role of `/` in the parameter list.
13. If a function has no return statement, what does it return? Why?
14. What is the difference between `print()` and `return` inside a function?
15. How can a function return multiple values? What is actually returned?
16. Is Python pass-by-value or pass-by-reference? Explain "pass by object reference".
17. Why does modifying a list inside a function affect the original list, but reassigning the parameter does not?
18. Explain the LEGB rule. In what order does Python resolve names?
19. What is the difference between local and global variables? Why does assigning to a global name inside a function create a new local variable?
20. What is the difference between the `global` and `nonlocal` keywords?
21. What is a closure? How does an inner function remember variables of its outer function?
22. What is the late-binding problem in closures and lambdas? How do you fix it?
23. What is a lambda function? What are its limitations compared to a normal function?
24. What is the difference between `lambda` and `def`? When would you prefer each?
25. How do `map()`, `filter()` and `reduce()` work? Why are comprehensions often preferred?

26. What is a recursive function? What is the base case, and why is it essential? What is Python's recursion limit and why does it exist?
27. What is a decorator? Explain how `@decorator` is translated internally, and why `functools.wraps` is used.
28. What is a generator function? How does `yield` differ from `return`, and how does it preserve state?
29. What are type hints and docstrings? Are type hints enforced at runtime? Where does Python store them?
30. What is the difference between a function and a callable object (`__call__`)? How do you check if something is callable?

Part 2: Coding Questions (1-10)

1. What will be the output? Why does this happen?

```
def add(a, b):  
    a + b  
  
print(add(2, 3))
```

2. Predict the output:

```
def f(x, L=[]):  
    L.append(x)  
    return L  
  
print(f(1))  
print(f(2))  
print(f(3, []))
```

3. What is the difference between these two calls? What is printed?

```
def show(*args, **kwargs):  
    print(args)  
    print(kwargs)  
  
show(1, 2, a=3)  
show((1, 2), {'a': 3})
```

4. Predict the output:

```
x = 10  
def change():  
    x = 20  
    print(x)  
  
change()  
print(x)
```

5. What will happen here?

```
x = 10
def change():
    print(x)
    x = 20

change()
```

6. Predict the output:

```
def modify(lst, n):
    lst.append(4)
    n += 1

a = [1, 2, 3]
b = 5
modify(a, b)
print(a, b)
```

7. What is the output? Explain the late-binding behaviour.

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])
```

8. Predict the output:

```
def outer():
    count = 0
    def inner():
        nonlocal count
        count += 1
        return count
    return inner

c = outer()
print(c(), c(), c())
```

9. What happens here?

```
def f(a, /, b, *, c):
    return a + b + c

print(f(1, 2, c=3))
print(f(1, b=2, c=3))
print(f(a=1, b=2, c=3))
```

10. Predict the output:

```
def gen():
    yield 1
    yield 2
    return 3

g = gen()
print(next(g))
print(next(g))
print(next(g))
```